

Data Structures - Sorting Techniques

Dr. Tayyaba Khushid

Department of Cyber Security,
Dawood University of Engineering and Technology (DUET).

September 7, 2026

Sorting Techniques

Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order.

Sorting Techniques

- Increasing Order
- Non-increasing order
- Decreasing order
- Non-Decreasing order

Increasing Order

A sequence of values is said to be in increasing order, if the successive element is greater than the previous one. For example, 1, 3, 4, 6, 8, 9 are in increasing order, as every next element is greater than the previous element.

Decreasing Order

A sequence of values is said to be in decreasing order, if the successive element is less than the current one. For example, 9, 8, 6, 4, 3, 1 are in decreasing order, as every next element is less than the previous element.

Non-Increasing Order

A sequence of values is said to be in non-increasing order, if the successive element is less than or equal to its previous element in the sequence. This order occurs when the sequence contains duplicate values. For example, 9, 8, 6, 3, 3, 1 are in non-increasing order, as every next element is less than or equal to (in case of 3) but not greater than any previous element.

Non-Decreasing Order

A sequence of values is said to be in non-decreasing order, if the successive element is greater than or equal to its previous element in the sequence. This order occurs when the sequence contains duplicate values. For example, 1, 3, 3, 6, 8, 9 are in non-decreasing order, as every next element is greater than or equal to (in case of 3) but not less than the previous one.

Sorting techniques

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Shell Sort
- Heap Sort
- Bucket Sort Algorithm
- Counting Sort Algorithm
- Quick Sort

Bubble sort

Bubble Sort Algorithm Bubble Sort is an elementary sorting algorithm, which works by repeatedly exchanging adjacent elements, if necessary. When no exchanges are required, the file is sorted.

We assume list is an array of n elements. We further assume that swap function swaps the values of the given array elements.

- Step 1 Check if the first element in the input array is greater than the next element in the array.
- Step 2 If it is greater, swap the two elements; otherwise move the pointer forward in the array.
- Step 3 Repeat Step 2 until we reach the end of the array.
- Step 4 Check if the elements are sorted; if not, repeat the same process (Step 1 to Step 3) from the last element of the array to the first.
- Step 5 The final output achieved is the sorted array.

</>

```
def bubble_sort(array, size):  
    for i in range(size):  
        swaps = 0;  
        for j in range(0, size-i-1):  
            if(arr[j] > arr[j+1]):  
                temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
                swaps = 1;  
        if(swaps == 0):  
            break;  
  
arr = [67, 44, 82, 17, 20]  
n = len(arr)  
print("Array before Sorting: ")  
print(arr)  
bubble_sort(arr, n);  
print("Array after Sorting: ")  
print(arr)
```

Figure: Bubble sorting Algo

Insertion sort

Insertion sort is a very simple method to sort numbers in an ascending or descending order. This method follows the incremental method

- Step 1 If it is the first element, it is already sorted. return 1;
- Step 2 Pick next element
- Step 3 Compare with all elements in the sorted sub-list
- Step 4 Shift all the elements in the sorted sub-list that is greater than the value to be sorted
- Step 5 Insert the value
- Step 6 Repeat until list is sorted

Insertion Sort Algorithm

```
def insertion_sort(array, size):  
    for i in range(1, size):  
        key = array[i]  
        j = i  
        while (j > 0) and (array[j-1] > key):  
            array[j] = array[j-1]  
            j = j-1  
        array[j] = key  
  
arr = [67, 44, 82, 17, 20]  
n = len(arr)  
print("Array before Sorting: ")  
print(arr)  
insertion_sort(arr, n);  
print("Array after Sorting: ")  
print(arr)
```

Figure: Insertion Sort

Selection Sort Algorithm

Selection Sort Algorithm This type of sorting is called Selection Sort as it works by repeatedly sorting elements. That is: we first find the smallest value in the array and exchange it with the element in the first position, then find the second smallest element and exchange it with the element in the second position, and we continue the process in this way until the entire array is sorted.

Selection Sort Algorithm

```
</>

def insertion_sort(array, size):
    for i in range(size):
        imin = i
        for j in range(i+1, size):
            if arr[j] < arr[imin]:
                imin = j
        temp = array[i];
        array[i] = array[imin];
        array[imin] = temp;

arr = [12, 19, 55, 2, 16]
n = len(arr)
print("Array before Sorting: ")
print(arr)
insertion_sort(arr, n);
print("Array after Sorting: ")
print(arr)
```

Figure: Caption

Merge Sort Algorithm

Merge sort is a sorting technique based on divide and conquer technique. Merge sort first divides the array into equal halves and then combines them in a sorted manner.

Merge Sort Algorithm

```
def merge_sort(a, n):  
    if n > 1:  
        m = n // 2  
        #divide the list in two sub lists  
        l1 = a[:m]  
        n1 = len(l1)  
        l2 = a[m:]  
        n2 = len(l2)  
        #recursively calling the function for sub lists  
        merge_sort(l1, n1)  
        merge_sort(l2, n2)  
        i = j = k = 0  
        while i < n1 and j < n2:  
            if l1[i] <= l2[j]:  
                a[k] = l1[i]  
                i = i + 1  
            else:  
                a[k] = l2[j]  
                j = j + 1  
            k = k + 1  
        while i < n1:  
            a[k] = l1[i]  
            i = i + 1  
            k = k + 1  
        while j < n2:  
            a[k] = l2[j]  
            j = j + 1  
            k = k + 1
```

Figure: Caption

Shell Sort Algorithm

Shell sort is a highly efficient sorting algorithm and is based on insertion sort algorithm. This algorithm avoids large shifts as in case of insertion sort, if the smaller value is to the far right and has to be moved to the far left.

Shell sort is a sorting method that improves how quickly we can sort things by comparing items that are far apart in the list, not just those next to each other.

Shell Sort Algorithm

```
def shell_sort(array,n):
    gap = n//2 #using floor division to avoid float values as result
    while gap > 0:
        for i in range(int(gap),n):
            temp = array[i]
            j = i
            while j >= gap and array[j-gap] >temp:
                array[j] = array[j-gap]
                j -= gap
            array[j] = temp
        gap = gap // 2 #using floor division to avoid float values as result

arr = [33, 45, 62, 12, 98]
n = len(arr)
print("Array before Sorting: ")
print(arr)
shell_sort(arr, n);
print("Array after Sorting: ")
print(arr)
```

Figure: Caption

Heap Sort Algorithm